

**Name: Shaikh Mohammed Saad**

**Roll no: B-39**

**Branch:- AI&DS(B)**

**Experiment-1 :-** Use of Sequence Data Types in Python

**Aim:-** To study and implement different sequence data types in Python and perform operations on them.

**Theory:-** Sequence data types are used to store multiple values in an ordered manner. They allow operations like indexing, slicing, iteration, and membership testing. Programs

**a) Strings**

```
s = "Hello Python"
print(s[0])
#Indexing
print(s[8:5])
# Slicing
print(s.lower())
# Operation
```

**b) Lists**

```
lst = 10, 20, 30, 40
lst.append(50)
print(lst)
print(lst [1:3])
```

**c) Arrays**

```
import array as arr
a = arr.array('i', [1, 2, 3, 4])
a.append(5)
print(a)
```

**d) Tuples**

```
t = (1, 2, 3, 4)
print(t[0])
print(t[1:3])
```

**f) Sets**

```
s = {1, 2, 3, 3, 4}
print(s)
# duplicates removed
```

**g) Range**

```
r = range(1, 10, 2)
print(list(r))
```

Double-click (or enter) to edit

**Theory Questions & Answers**

**1. What are sequence data types in Python?**

Sequence data types store multiple values in an ordered way and allow indexing and slicing.

**Types:**

- String
- List
- Tuple
- Range
- Array (from module)

**Indexing & Slicing** Indexing: Access element using position Example: `s[0]` → "rst element"

Slicing: Access a range Example: `s[1:4]` → elements from index 1 to 3

✓ Works similarly in strings, lists, tuples, and arrays

✓ Syntax: `[start : end : step]`

## 2. Difference Between List and Tuple

| Feature     | List             | Tuple            |
|-------------|------------------|------------------|
| Mutability  | Mutable          | Immutable        |
| Syntax      | <code>[ ]</code> | <code>( )</code> |
| Use Case    | Dynamic data     | Fixed data       |
| Performance | Slower           | Faster           |

Why prefer tuple?

Data safety (cannot be changed)

Faster execution

Used in constants (e.g., coordinates)

## 3. Arrays vs Lists

| Feature     | List                                | Array (from <code>array</code> module)                    |
|-------------|-------------------------------------|-----------------------------------------------------------|
| Data Type   | Stores heterogenous data types      | Stores homogenous data types                              |
| Flexibility | More flexible, can grow/shrink      | Fixed type after creation                                 |
| Memory      | More memory overhead                | More memory efficient                                     |
| Operations  | General-purpose operations          | Optimized for numerical operations                        |
| Usage       | Common for general data collections | Primarily for numeric data, e.g., when interfacing with C |
| Syntax      | <code>[item1, item2, ...]</code>    | <code>array('typecode', [item1, ...])</code>              |

### When to use arrays?

1. When handling large numeric data
2. When memory efficiency is important

## 4. How Dictionaries Work Internally

- Dictionaries use hashing
- Each key is converted into a hash value
- Data is stored using key-value pairs

## 5. Difference Between Sets and Lists

| Feature          | List                                   | Set                                                       |
|------------------|----------------------------------------|-----------------------------------------------------------|
| Order            | Ordered                                | Unordered                                                 |
| Duplicates       | Allows duplicates                      | Does not allow duplicates                                 |
| Mutability       | Mutable (can change elements)          | Mutable (can add/remove elements)                         |
| Indexing/Slicing | Supported                              | Not supported                                             |
| Syntax           | <code>[element1, element2, ...]</code> | <code>{item1, item2, ...}</code> (or <code>set()</code> ) |
| Use Case         | Collection of items, ordered sequences | Unique collections, membership testing                    |

### Handling Duplicates

- Sets automatically remove duplicates

### Practical Use

- Removing duplicates from data
- Membership testing
- Mathematical operations (union, intersection)

### Conclusion:-

In this assignment, we learned about different sequence data types in Python such as strings, lists, arrays, tuples, dictionaries, sets, and range. We also performed operations like indexing and slicing, and understood their real-world applications.

